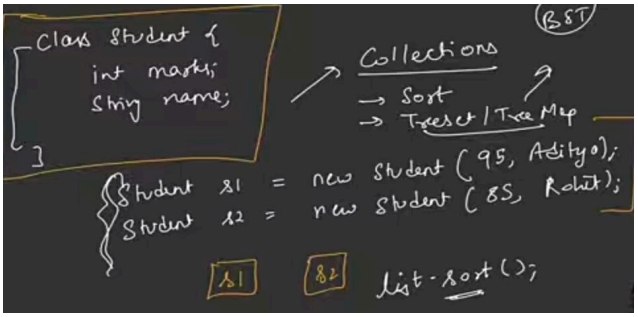
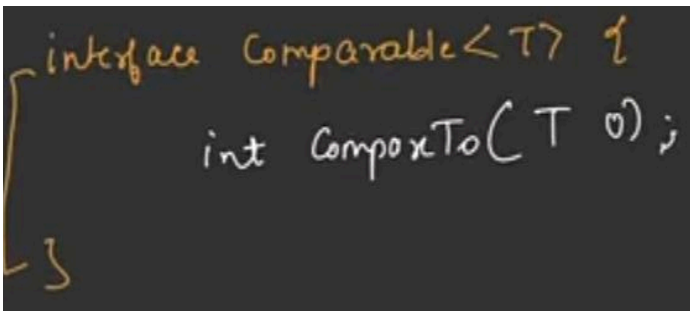
Comparable Interface in java….

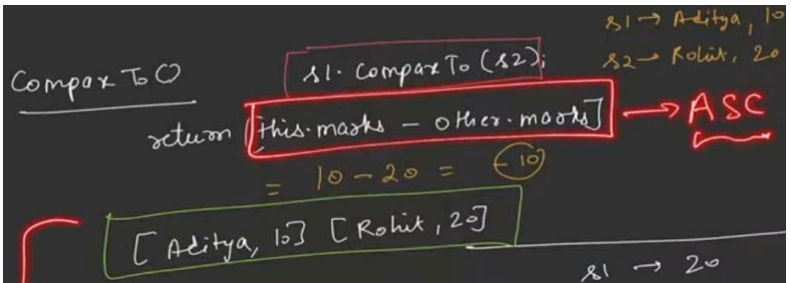


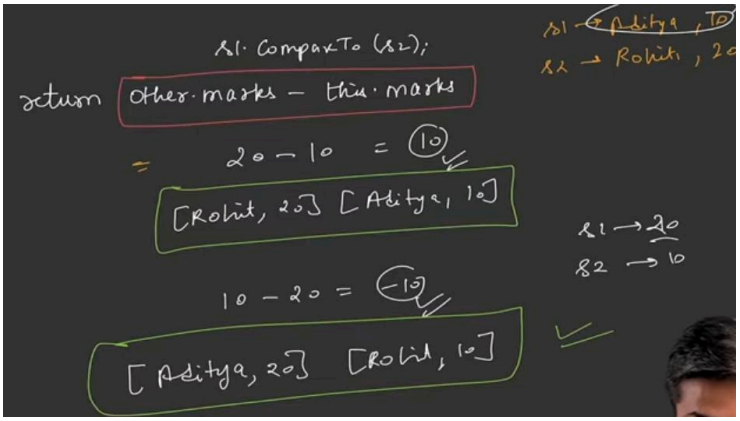


- It is an interface so it has a contract that, I know how to compare myself with another object of my type.
- This interface is a **functional interface** because it have only one method.

```
    public static void main(String[] args) {
        List<Integer> list2 = new ArrayList<>();
            list2.add(10);
            list2.add(34);
            list2.add(3);
            list2.add(24 );
                Collections.sort(list2);
                System.out.println(list2);// Ans--> [3, 10, 24, 34]
        // but here i can't do this.
        List<Student> list = new ArrayList<>();
            list.add(new Student("Priyansh", 95));
            list.add(new Student("Rahul", 85));
            list.add(new Student("Raghav", 92));
            list.add(new Student("Arvind", 85 ));
                Collections.sort(list);//asscending order. I am not able to perform this action.
        // so, i have to implement the comparable interface below-->
                for(Student s : list){
                    System.out.println(s.name + ", " + s.marks);
                }
    }
}
class Student implements Comparable<Student>{
    String name;
    int marks;
    public Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
    // @Override
    //     public int compareTo(Student other){
    //         return this.marks - other.marks;// for Asscending order Comparison.
    //         //return other.marks - this.marks; // for descending order Comparison.
    //     }
    //Lexicographical order comparison-->
    @Override
        public int compareTo(Student other){
            if(this.marks != other.marks){
                return this.marks - other.marks;
            }
            return this.name.compareTo(other.name);
        }
}
```

Internal process behind this code above—^





- In Student class if we want to **sort**, Treeset/TreeMap we use the **compareTo();** method.
- In java almost all **in-built** classes uses **comparable Interface** directly or indirectly.

Que → Why Comparable Interface gives only one natural ordering ?

Ans → java is not able to sort between different-different natural orderings, therefore one class should have only one natural ordering.

- The reason behind **int→ in compareTo method** is we want result in **(-ve), (+ve), (0).**

Dangers of 0 return

- **Jab bhi compare to 0 return krta ha to vo 0 ane pr values ko similar nahi manta blki unhe equal manta ha, jiska mtlb uske liye dono values same ha.**
  Or set & map me equal values ko repeat nahi kiya jata ha, because duplicates are not allowed in these dataStructures, or is wajah se values unme add ni ho pati ha.

Ans→ to overcome this situation, hum khud se btana pdega ki isko equal hone k liye iske lexicographical order bhi equal hone chahiye.

COLLECTIONS CLASS

- **This is a utility class.**
- **It have several static methods & so we have no need to make their objects, we can call directly.**
- **This class can apply on any collection → set, map, list, etc.**
- **So to call static method we have to only do → (collections.)**
  Collections.sort()
  Collections.max()
  Collections.min()
  Collections.fill()
  Collections.rverse()
  Collections.shuffle()
  Collections.swap(list, a, b)
  Collections.binarySearch()
  Collections.frequwency(list, 5);
  Collections.unmodifiablelist(list)-->this will return immutable read-only LIST.
  Collections.emptylist()-->this will also return immutable and read-only LIST.